



05201302

Web Application Development

MCA MSc-IT Semester 3

Prof Faruk Abdulla
Assistant Professor PIET - MCA



Unit - 4

Content

1. Basic features of ADO.NET
1. ADO.NET Architecture
1. Using SQL data sources
1. Grid view control
1. Detail view control
1. Form view control
1. List view and other controls
1. Using object data sources



Introduction to ADO.NET

ADO.NET (ActiveX Data Objects for .NET) is a data access technology provided by the .NET Framework for **connecting, retrieving, manipulating, and updating** data from **various data sources** like SQL Server, Oracle, XML, or OLE DB-compliant sources.

It provides a **bridge** between front-end ASP.NET applications and back-end databases.



Key Definitions:

ADO.NET: A set of classes in the System.Data namespace used to interact with databases.

Data Source: The database or file where data is stored (e.g., SQL Server).

Data Provider: A collection of classes that enable communication with a specific database (e.g., SqlConnection for SQL Server).



Basic Features of ADO.NET

Disconnected Data Access

In classic ADO (before .NET), applications used a connected model (continuous connection to the database). This caused high resource consumption when multiple users accessed the system.

In ADO.NET, data can be retrieved once, stored in memory (using DataSet or DataTable), and then worked upon offline without requiring a continuous connection.

This makes applications highly scalable and reduces load on the database server.



Basic Features of ADO.NET

Scalability

- Since ADO.NET uses disconnected architecture, thousands of users can access data without each needing a permanent database connection.
- Only short-lived connections are required for fetching and updating data.
- This is especially useful for web applications, where multiple users may be working simultaneously.

Example: A university portal with 10,000 students checking results at the same time. With disconnected architecture, the server only keeps connections open briefly for each request.



Basic Features of ADO.NET

Interoperability with XML

- ADO.NET integrates deeply with XML (Extensible Markup Language).
- DataSet internally uses XML format to store and transfer data.
- Supports reading/writing XML directly with methods like:
 - ReadXml()
 - WriteXml()

Use Case:

- Sharing data between different applications/platforms.
- For example, sending student attendance data from a .NET app to a Java-based system.



Basic Features of ADO.NET

Strongly Typed Classes

- ADO.NET provides strongly typed datasets where each column can be accessed as a property of an object.
- This eliminates errors caused by misspelled column names or wrong data types.
- Helps in type safety and compile-time checking.



Basic Features of ADO.NET

Multiple Data Providers

ADO.NET provides different data providers to interact with different databases. Each provider includes classes like Connection, Command, DataReader, and DataAdapter.

Main Providers:

- **SqlClient** – optimized for Microsoft SQL Server (Namespace: System.Data.SqlClient)
- **OleDb** – for OLE DB-supported data sources like MS Access (Namespace: System.Data.OleDb)
- **Odbc** – for ODBC-supported databases (Namespace: System.Data.Odbc)
- **OracleClient** – for Oracle databases (though deprecated now) (Namespace: System.Data.OracleClient)



Basic Features of ADO.NET

DataSet for Offline Storage

- A DataSet is an in-memory representation of data that can store:
 - Multiple DataTables
 - Relationships (DataRelation)
 - Constraints (Primary Key, Foreign Key)
- Works in disconnected mode.

DataReader for Fast Access

- DataReader provides a fast, forward-only, read-only way to fetch data.
- Consumes fewer resources compared to DataSet.



Basic Features of ADO.NET

Integration with Web Controls

- ADO.NET integrates with ASP.NET data-bound controls like:
 - GridView, DetailsView, FormView, ListView
- Makes it easy to bind data sources to controls without writing lengthy code.

Support for Transactions

- ADO.NET supports database transactions using SqlTransaction.
- Ensures atomicity (all-or-nothing execution of queries).

Basic Features of ADO.NET

Feature	Description	Use Case
Disconnected Access	Work offline with DataSet	Student report, survey forms
Scalability	Thousands of users supported	University/Bank portals
XML Support	Import/export in XML	Data sharing between apps
Strongly Typed	Safer coding with datasets	Enterprise systems
Multiple Providers	SQL, Oracle, Access, ODBC	Flexible connections
DataReader	Fast, forward-only access	Large result sets
DataSet	Offline, relational data cache	Multi-table data manipulation
Web Control Integration	Easy data binding	ASP.NET GridView, FormView
Transactions	Safe commit/rollback	Bank transactions



ADO.NET Architecture

ADO.NET follows a two-tier architecture:

Data Provider (Connected Layer)

- Responsible for connecting to the database, executing commands, and retrieving results.
- Classes include:
 - SqlConnection – establishes connection to SQL Server
 - SqlCommand – executes SQL statements/stored procedures
 - SqlDataReader – fast, forward-only reading of data
 - SqlDataAdapter – acts as a bridge between database and DataSet

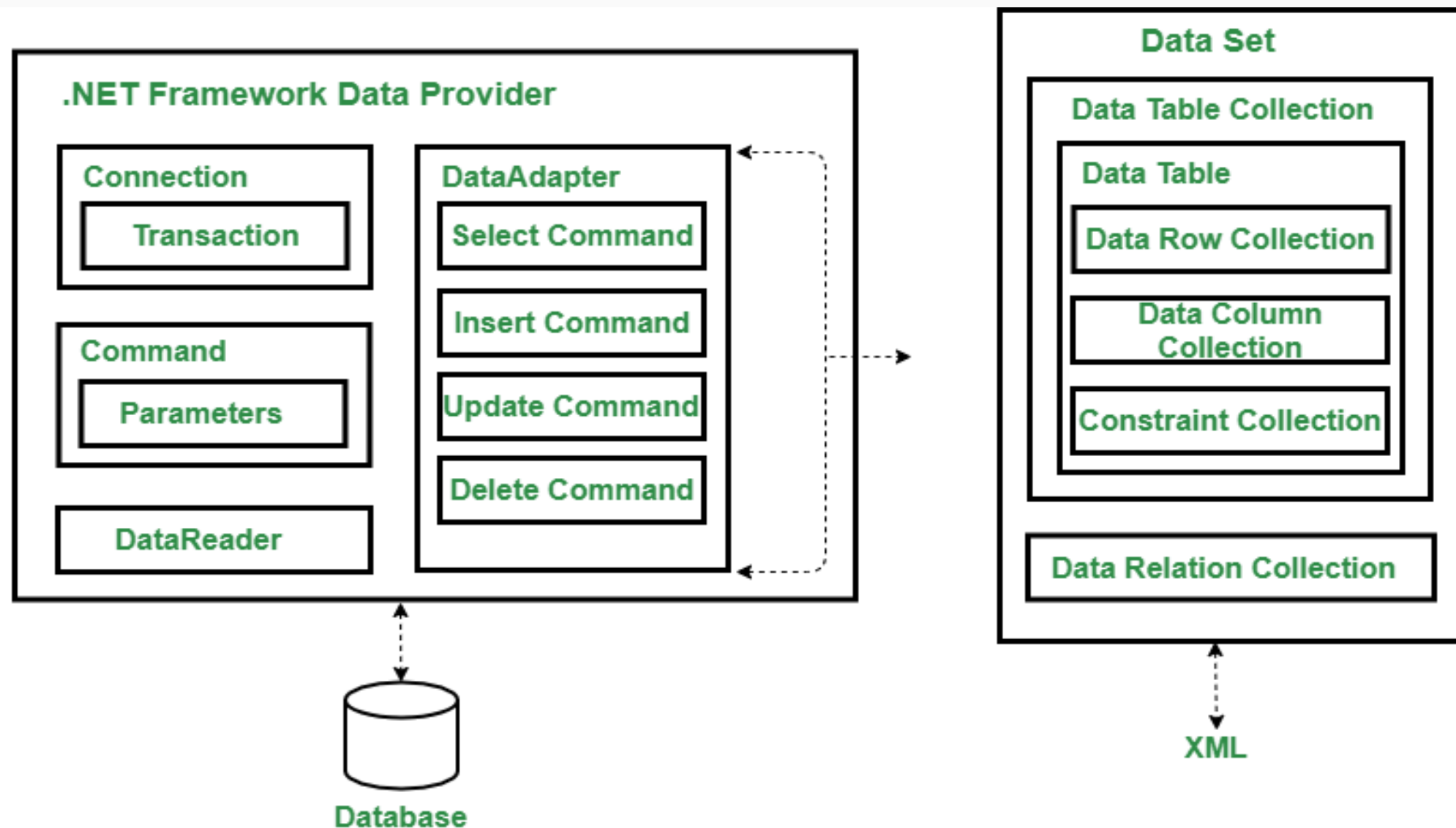


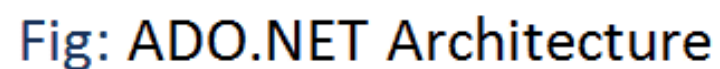
ADO.NET Architecture

DataSet (Disconnected Layer)

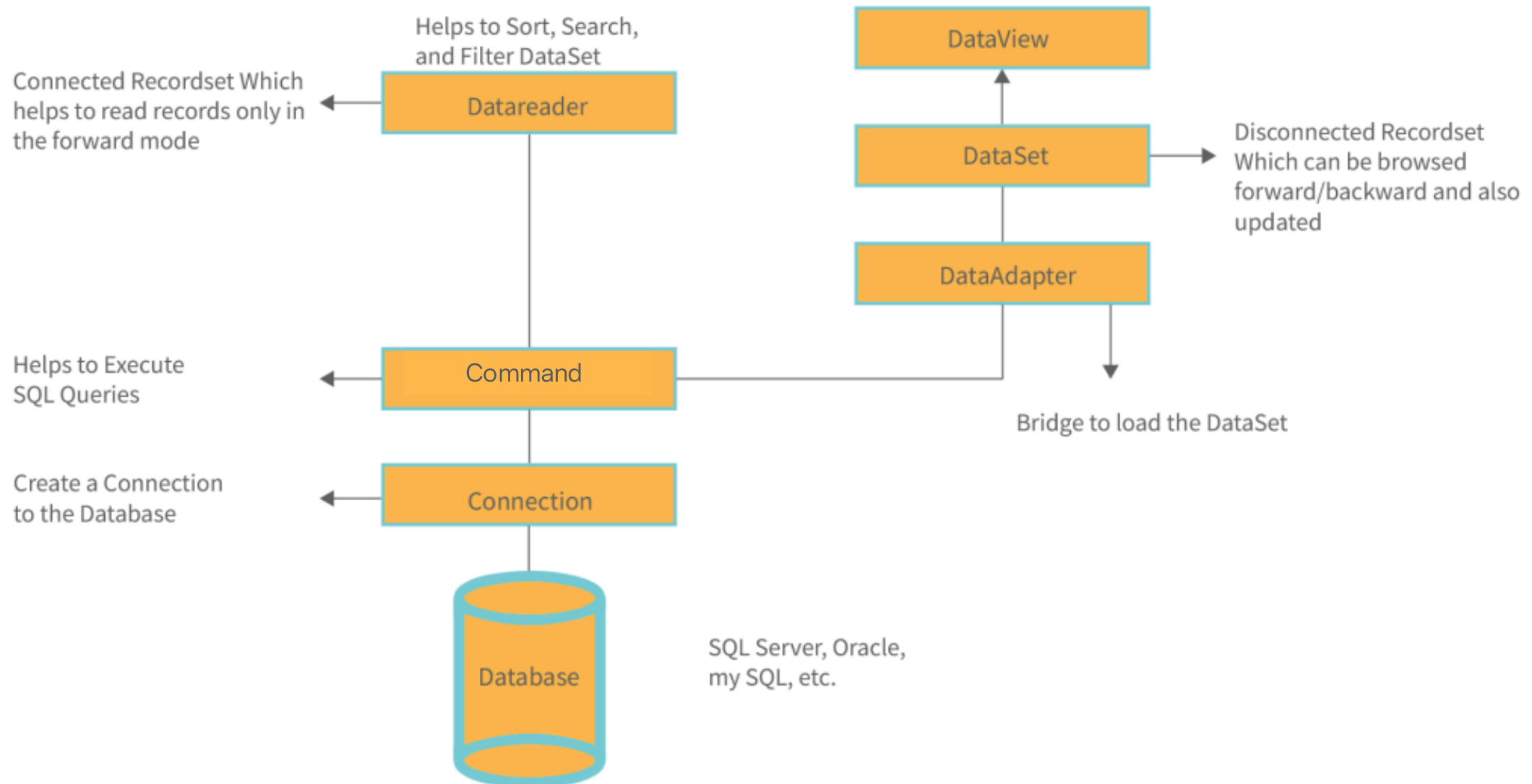
- An in-memory cache of data.
- Stores multiple tables (DataTable), relationships (DataRelation), and constraints.
- Works offline (no continuous database connection).
- Useful in web applications where scalability is required.

ADO.NET Architecture





ADO.NET Architecture





Using SQL Data Sources

- The SqlDataSource control in ASP.NET provides a declarative way (no/less C# code required) to connect to a SQL database and perform operations like Select, Insert, Update, and Delete.
- It acts as a bridge between the database and data-bound controls like GridView, DetailsView, ListView, DropDownList.



Using SQL Data Sources

- Provides built-in support for:
 - Connecting to the database.
 - Executing SQL commands.
 - Automatically binding data to UI controls.
- Eliminates the need to write explicit connection and command code (though you can still use it for complex logic).

Using SQL Data Sources

Property	Description	Example
ID	Unique identifier of control	ID="SqlDataSource1"
ConnectionString	Database connection string	<%\$ ConnectionStrings:MyDB %>
ProviderName	Database provider name	System.Data.SqlClient
SelectCommand	SQL query for retrieving data	SELECT * FROM Students
InsertCommand	SQL query for inserting data	INSERT INTO Students(Name, RollNo) VALUES(@Name, @RollNo)
UpdateCommand	SQL query for updating data	UPDATE Students SET Name=@Name WHERE RollNo=@RollNo
DeleteCommand	SQL query for deleting data	DELETE FROM Students WHERE RollNo=@RollNo
FilterExpression	Allows filtering records	"Course='BCA'"
SelectParameters	Parameters for Select query	<SelectParameters> tag

Using SQL Data Sources

Using SqlDataSource with GridView

```
<asp:SqlDataSource ID="SqlDataSource1" runat="server"  
    ConnectionString="<%%$ ConnectionStrings:MyDBConnection %>"  
    SelectCommand="SELECT * FROM Students">  
</asp:SqlDataSource>
```

```
<asp:GridView ID="GridView1" runat="server"  
    DataSourceID="SqlDataSource1" AutoGenerateColumns="true">  
</asp:GridView>
```



Using SQL Data Sources

Performing CRUD Operations

```
<asp:SqlDataSource ID="SqlDataSource1" runat="server"
  ConnectionString="<%%$ ConnectionStrings:MyDBConnection %>"
  SelectCommand="SELECT * FROM Students"
  InsertCommand="INSERT INTO Students (Name, RollNo) VALUES (@Name,
@RollNo)"
  UpdateCommand="UPDATE Students SET Name=@Name WHERE
RollNo=@RollNo"
  DeleteCommand="DELETE FROM Students WHERE RollNo=@RollNo">

<InsertParameters>
  <asp:Parameter Name="Name" Type="String" />
  <asp:Parameter Name="RollNo" Type="Int32" />
</InsertParameters>
```

Using SQL Data Sources

```
<UpdateParameters>  
  <asp:Parameter Name="Name" Type="String" />  
  <asp:Parameter Name="RollNo" Type="Int32" />  
</UpdateParameters>  
  
<DeleteParameters>  
  <asp:Parameter Name="RollNo" Type="Int32" />  
</DeleteParameters>  
</asp:SqlDataSource>  
  
<asp:GridView ID="GridView1" runat="server"  
  DataSourceID="SqlDataSource1" AutoGenerateColumns="true"  
  AutoGenerateEditButton="true" AutoGenerateDeleteButton="true">  
</asp:GridView>
```



Using SQL Data Sources

Filtering Data using Parameters

You can pass parameters dynamically using:

- QueryStringParameter (from URL)
- ControlParameter (from a control like TextBox/DropDown)
- SessionParameter
- FormParameter

Using SQL Data Sources

```
<asp:TextBox ID="txtCourse" runat="server"></asp:TextBox>
<asp:Button ID="btnSearch" runat="server" Text="Search" />

<asp:SqlDataSource ID="SqlDataSource1" runat="server"
    ConnectionString="<%%$ ConnectionStrings:MyDBConnection %>"
    SelectCommand="SELECT * FROM Students WHERE Course=@Course">
    <SelectParameters>
        <asp:ControlParameter ControlID="txtCourse" Name="Course"
            PropertyName="Text" />
    </SelectParameters>
</asp:SqlDataSource>

<asp:GridView ID="GridView1" runat="server"
    DataSourceID="SqlDataSource1"></asp:GridView>
```


Using SQL Data Sources

Using Stored Procedures

```
<asp:SqlDataSource ID="SqlDataSource1" runat="server"
  ConnectionString="<%%$ ConnectionStrings:MyDBConnection %>"
  SelectCommand="GetStudentsByCourse"
  SelectCommandType="StoredProcedure">
  <SelectParameters>
    <asp:Parameter Name="CourseName" Type="String" DefaultValue="BCA" />
  </SelectParameters>
</asp:SqlDataSource>
```



Using SQL Data Sources

Security Considerations

- Always use parameterized queries (@Name, @RollNo) to prevent SQL Injection.
- Use Stored Procedures when possible. Keep connection strings in Web.config (never hardcoded).

Advantages of SqlDataSource

- Minimal coding (drag-and-drop friendly)
- Supports CRUD operations out of the box
- Easy to bind with GridView, DetailsView, FormView
- Supports filtering and parameters

Limitations

- Not suitable for very complex queries or business logic
- Tight coupling of UI and Database (less flexible in enterprise apps)
- Harder to debug if queries fail

GridView Control

- Definition GridView is the most widely used data-bound control in ASP.NET.
- It displays data in a tabular (row and column) format.
- Supports sorting, paging, editing, deleting, and selection of rows.

Property	Description
DataSourceID	ID of the data source control (e.g., SqlDataSource).
AutoGenerateColumns	If true, columns are created automatically.
AutoGenerateEditButton	Adds an Edit button in each row.
AutoGenerateDeleteButton	Adds a Delete button in each row.
AllowPaging	Enables paging of records.
AllowSorting	Enables sorting of records.
Columns	Allows defining custom BoundField, TemplateField, etc.

GridView Control

```
<asp:GridView ID="GridView1" runat="server"  
    DataSourceID="SqlDataSource1"  
    AutoGenerateColumns="true"  
    AllowPaging="true" PageSize="5"  
    AllowSorting="true">  
</asp:GridView>
```

```
<asp:SqlDataSource ID="SqlDataSource1" runat="server"  
    ConnectionString="<%$  
ConnectionStrings:MyDBConnection %>"  
    SelectCommand="SELECT * FROM Students">  
</asp:SqlDataSource>
```

GridView Control

UID	Name	Surname	City	Email	Password
1	Meera	Patel	Surat	meera@yahoo.com	123
2	Jay	Patel	surat	jay@yahoo.com	12345
3	Mehul	Patel	Ahmedabad	mehul@yahoo.com	mehul123
4	Shital	Patel	Ahmedabad	shital@yahoo.com	s123
5	Vaidehi	Patel	Baroda	vaidehi@yahoo.com	vaidehi123
6	Dharv	Patel	Baroda	dharv@yahoo.com	d123

DetailsView Control

- Displays data in a vertical table where each row shows a field name and its value.
- Useful for showing details of a single record.
- Supports Insert, Edit, Delete.

Property	Description
AutoGenerateRows	Automatically creates rows for each field.
DataSourceID	ID of SqlDataSource/ObjectDataSource

DetailsView Control

```
<asp:DetailsView ID="DetailsView1" runat="server"  
    DataSourceID="SqlDataSource1"  
    AutoGenerateRows="true"  
    AutoGenerateEditButton="true"  
    AutoGenerateDeleteButton="true">  
</asp:DetailsView>
```

```
<asp:SqlDataSource ID="SqlDataSource1" runat="server"  
    ConnectionString="<%$  
ConnectionStrings:MyDBConnection %>"  
    SelectCommand="SELECT * FROM Students WHERE  
RollNo=1">
```

DetailsView Control

Student Information	
Name	abc
DOB	12/3/2010 12:00:00 AM
Age	0
Course	abc
Branch	abc
City	abc
MobileNo	abc
1 <u>2</u>	



FormView Control

- Similar to DetailsView but more flexible.
- Allows full customization of layout using templates.
- Useful when you want complete control over HTML structure.

Property	Description
ItemTemplate	Template used to display a record.
EditItemTemplate	Template for editing a record.



FormView Control

```
<asp:FormView ID="FormView1" runat="server" DataSourceID="SqlDataSource1">
  <ItemTemplate>
    <h3>Student Information</h3>
    Name: <%# Eval("Name") %><br />
    Roll No: <%# Eval("RollNo") %><br />
    Course: <%# Eval("Course") %><br />
  </ItemTemplate>
  <EditItemTemplate>
    Name: <asp:TextBox ID="txtName" runat="server" Text='<%# Bind("Name") %>' /><br />
    Course: <asp:TextBox ID="txtCourse" runat="server" Text='<%# Bind("Course") %>' /><br />
    <asp:Button ID="btnUpdate" runat="server" CommandName="Update" Text="Update" />
  </EditItemTemplate>
</asp:FormView>

<asp:SqlDataSource ID="SqlDataSource1" runat="server"
  ConnectionString="<%$ ConnectionStrings:MyDBConnection %>"
  SelectCommand="SELECT * FROM Students WHERE RollNo=1"
  UpdateCommand="UPDATE Students SET Name=@Name, Course=@Course WHERE RollNo=@RollNo">
</asp:SqlDataSource>
```


FormView Control

Student Information

StudentID: Databound
StudentName: Databound
DOB: Databound
Age: Databound
Course: Databound
City: Databound
MobileNo: Databound



ListView Control

- A highly flexible data-bound control.
- Allows full customization of layout (list, table, grid, gallery, etc.).
- Unlike GridView, it does not enforce tabular format.
- Can be used with DataPager for paging.

Property	Description
ItemTemplate	Template for each item.
AlternatingItemTemplate	Template for alternate items.
	Defines the overall

Listview Control

```
<asp:ListView ID="ListView1" runat="server" DataSourceID="SqlDataSource1">
  <LayoutTemplate>
    <div class="student-list">
      <asp:Placeholder ID="itemPlaceholder" runat="server"></asp:Placeholder>
    </div>
  </LayoutTemplate>
  <ItemTemplate>
    <div class="student-item">
      <b><%# Eval("Name") %></b> (Roll No: <%# Eval("RollNo") %>)<br />
      Course: <%# Eval("Course") %><br /><hr />
    </div>
  </ItemTemplate>
</asp:ListView>
<asp:SqlDataSource ID="SqlDataSource1" runat="server"
  ConnectionString="<%= $ ConnectionStrings.MyDBConnection %>"
  SelectCommand="SELECT * FROM Students">
</asp:SqlDataSource>
```



ListView Control

Kevin Brown

Marketing Assistant
kevin0@adventure-works.com
150-555-0189

7883 Missing Canyon Court
Everett, Washington 98201
United States

Jo Brown

Production Supervisor - WC60
jo0@adventure-works.com
632-555-0129

9693 Mellowood Street
Duvall, Washington 98019
United States

Janaina Bueno

Application Specialist
janaina0@adventure-works.com
623-555-0155

5979 El Pueblo
Issaquah, Washington 98027
United States

<-- 11 12 13 14 15 -->



Quick Comparison

Control	Layout	Best For
GridView	Tabular (rows & columns)	Student records, employee lists, product lists
DetailsView	Vertical (one record at a time)	Profile details, single record operations
FormView	Fully customizable	Custom profile pages, detailed product views
ListView	Flexible (list, grid, gallery)	Catalogs, student lists, employee directories



Using object data sources

- ObjectDataSource is a data source control that allows us to bind UI controls (like GridView, DetailsView, FormView, ListView) to custom business objects or middle-layer classes, instead of directly binding to a database.
- It is part of N-tier (3-tier) architecture where:
 - Presentation Layer (UI — ASPX page)
 - Business Logic Layer (BLL) — C# class that contains methods for data operations.
 - Data Access Layer (DAL) — C# class that interacts with the database.
- ObjectDataSource acts as a bridge between UI controls and business objects.



Using object data sources

Key Features

- Decouples UI from database — promotes separation of concerns.
- Works with methods of classes instead of SQL queries.
- Supports Select, Insert, Update, and Delete methods.
- Enables reusability of business logic classes.
- Makes applications more scalable and testable.

Using object data sources

Property	Description
TypeName	The name of the business object (class) to use.
SelectMethod	The method that retrieves data.
InsertMethod	The method that inserts a record.
	The method that updates



Using object data sources

Advantages of ObjectDataSource

- Separation of Concerns – UI and data logic are independent.
- Reusability – Business logic can be reused in other applications (e.g., Web API).
- Testability – Easy to test business logic separately.
- Scalability – Works well in enterprise N-tier applications.
- Flexibility – Can work with different databases (since DAL is abstracted).

Real-Life Use Cases

- Student Management System (UI binds to StudentManager instead of direct DB).